

CyLab: Heap

Nguyễn Lê Anh Tuấn

Kiến thức nền tảng về Heap

Heap là vùng bộ nhớ được cấp phát *động* trong thời gian thực thi chương trình, thông qua các hàm như `malloc()`, `calloc()`, `realloc()` và được giải phóng bằng `free()`. Khác với stack – nơi các biến cục bộ được tạo và hủy tự động theo từng lời gọi hàm – heap cung cấp vùng nhớ tồn tại xuyên suốt vòng đời của chương trình cho đến khi được giải phóng rõ ràng.

Cấu trúc chunk trên heap (glibc)

Trong triển khai glibc (GNU C Library), mỗi vùng nhớ được cấp phát qua `malloc()` được gọi là một *chunk*. Mỗi chunk bao gồm phần metadata (tiêu đề) và phần dữ liệu người dùng:

Trường	Kích thước (64-bit)
<code>prev_size</code>	8 byte
<code>size (+ flags)</code>	8 byte
<i>user data...</i>	<i>n byte</i>

Con trỏ trả về bởi `malloc()` trỏ vào phần *dữ liệu người dùng*, không phải vào phần metadata. Vì vậy, khi hai lần cấp phát liên tiếp được thực hiện, con trỏ thứ hai sẽ cách con trỏ thứ nhất một khoảng đúng bằng kích thước chunk đầy đủ (bao gồm metadata và padding căn chỉnh).

Lỗi Heap Overflow xảy ra như thế nào

Heap overflow xảy ra khi chương trình ghi dữ liệu vào một vùng nhớ trên heap vượt quá kích thước đã được cấp phát, làm tràn sang vùng nhớ kề bên. Điều này thường xảy ra khi:

- Dùng `scanf("%s", ptr)` mà không giới hạn độ dài đầu vào
- Dùng `strcpy()` không kiểm tra kích thước chuỗi nguồn
- Tính toán sai kích thước buffer cần cấp phát

Khi hai vùng nhớ liền kề trên heap, dữ liệu tràn từ vùng đầu tiên có thể ghi đè lên nội dung của vùng thứ hai – bao gồm cả metadata chunk, con trỏ hàm, hoặc chuỗi dữ liệu quan trọng.

Dấu hiệu nhận biết bài Heap

- Chương trình có menu dạng: Create / Edit / Delete / View
- Có sử dụng `malloc()` và `free()`
- Có thể tạo nhiều “object” với kích thước khác nhau
- Cho phép thao tác sau khi xóa (use-after-free)
- Có hàm ẩn như `win()` không bao giờ được gọi trực tiếp

file và checksec trong bài Heap

Trước khi bắt tay vào khai thác một bài Heap, việc phân tích thông tin tệp và các cơ chế bảo vệ là bắt buộc, vì nó định hình toàn bộ chiến thuật:

1. Lệnh file (Xác định môi trường cơ bản)

- **Kiến trúc (32-bit vs 64-bit):** Ảnh hưởng trực tiếp đến kích thước metadata của chunk. Trên 32-bit, metadata thường là 8 byte; trên 64-bit là 16 byte. Kích thước con trỏ (pointer) để đếm offset cũng thay đổi tương ứng (4 byte vs 8 byte).
- **Liên kết (Dynamically vs Statically linked):** Quyết định việc ta có thể ghi đè bảng GOT (Global Offset Table) hay không. Nếu liên kết tĩnh (statically linked), bảng GOT thường không tồn tại hoặc không hữu dụng, ta phải tìm các mục tiêu khác.

2. Lệnh checksec (Định hình hướng tấn công)

- **RELRO:** Đây là cờ quan trọng nhất trong các bài Heap. Nếu là *Partial RELRO*, mục tiêu phổ biến là dùng lỗ hổng Heap (ví dụ: Arbitrary Write) để ghi đè bảng GOT (thay đổi `free@GOT` thành địa chỉ của `system`). Nếu là *Full RELRO*, bảng GOT bị khóa (chỉ đọc), ta buộc phải nhắm vào các mục tiêu khác như `__malloc_hook`, `__free_hook` (trong libc cũ) hoặc ghi đè Return Address trên Stack.
- **PIE (Position Independent Executable):** Khi kết hợp PIE với ASLR của hệ thống, địa chỉ của chương trình và địa chỉ Heap bị xáo trộn. Ta thường phải tìm lỗ hổng rò rỉ (leak) được hai loại địa chỉ: *Heap base* (để tính toán vị trí các chunk) và *libc base* (để gọi ROP hoặc lấy địa chỉ `system`).
- **NX (Non-Executable):** NX ngăn chặn việc đặt trực tiếp shellcode lên một chunk trên Heap rồi nhảy đến đó thực thi. Buộc ta phải sử dụng kỹ thuật ROP hoặc Ret2libc.

Mục tiêu khai thác phổ biến

- Ghi đè nội dung biến quan trọng (chuỗi, flag kiểm tra)
- Ghi đè function pointer để điều hướng luồng thực thi (thông qua GOT Overwrite hoặc Hook Overwrite).
- Kiểm soát địa chỉ mà `malloc()` trả về (Heap metadata corruption, tcache poisoning, fastbin dup...).